

PrepEdge – Interview Preparation Platform

Project Booklet (Hows & Whys)

1. Project Overview

Project Name: PrepEdge

Type: Web Application

Domain: Interview Preparation / EdTech

Deployment: Cloud-hosted (Render)

PrepEdge is a role-based interview preparation platform designed to help aspirants practice interview questions through mock interviews and a structured question bank, while allowing admins to manage questions.

The project focuses on **core backend concepts**, clean architecture, and real-world deployment rather than unnecessary complexity.

2. Problem Statement

Many students prepare for interviews using scattered resources (notes, random websites, PDFs) without a structured practice environment. There is often: - No role separation (admin vs learner) - No controlled question management - No hands-on mock interview experience

PrepEdge solves this by: - Centralizing interview questions - Allowing role-based access - Providing a mock interview flow

3. Target Users

3.1 Aspirant

- Can sign up and log in
- Can access dashboard
- Can attempt mock interviews
- Can explore the question bank

3.2 Admin

- Can log in like a normal user
- Has elevated privileges

- Can add interview questions to the database
- Controls content, not users

Admin roles are assigned manually to avoid privilege misuse.

4. Technology Stack (What Was Used)

4.1 Backend

- **Python** – Core programming language
- **Flask** – Lightweight web framework

Why Flask? - Beginner-friendly - Minimal and flexible - Ideal for learning backend fundamentals - No unnecessary abstraction

4.2 Frontend

- **HTML** – Page structure
- **CSS** – Styling and layout

Why not React or frameworks? - Focus was backend logic - HTML/CSS is sufficient for project scope - Avoided overengineering

4.3 Database

- **SQLite** – Relational database

Why SQLite? - Simple setup - File-based (no server needed) - Ideal for small to medium projects - Easy to demonstrate locally

4.4 Version Control

- **Git** – Local version control
- **GitHub** – Remote repository hosting

Why GitHub? - Industry standard - Enables collaboration - Required for cloud deployment

4.5 Deployment

- **Render (Web Service)**

Why Render? - Native Python support - Suitable for Flask apps - Free tier available - Easier than AWS for beginners

5. Technologies NOT Used (And Why)

✗ Django

- Too heavy for project scope
- Requires prior framework knowledge

✗ React / Next.js

- Frontend complexity not required
- Backend learning was priority

✗ Vercel

- Optimized for frontend apps
- Does not support long-running Flask servers

✗ PostgreSQL

- Overkill for project size
 - SQLite sufficient
-

6. Project Structure (How It Is Organized)

```
PrepEdge/
|
├── app.py                # Main Flask application
├── database_setup.py     # Database schema creation
├── make_admin.py         # Admin role assignment script
├── requirements.txt      # Dependencies for deployment
├── README.md             # Project documentation
|
├── templates/           # HTML templates
|   ├── home.html
|   ├── login.html
|   ├── signup.html
|   ├── dashboard.html
|   ├── mock.html
|   ├── questions.html
|   └── add-question.html
|
├── static/
|   └── css/
|       └── style.css
```



venv/

Virtual environment (ignored in Git)

7. Application Flow (How It Works)

7.1 Home Page (/)

- Entry point of the application
- Provides navigation to login/signup

7.2 Authentication

Signup

- User submits username & password
- Stored in database
- Default role:

Login

- Credentials verified
- Session created
- User role stored in session

7.3 Session Management

- Flask sessions used
- Session stores:
 - username
 - role (admin / aspirant)

Why sessions? - Stateless HTTP needs state handling - Sessions enable secure login persistence

7.4 Dashboard

Displayed after login.

Aspirant sees:

- Start Mock Interview
- Explore Questions

Admin sees:

- Add Question

Dashboard content is **role-based**, not route-based.

7.5 Mock Interview

- Questions fetched from database
- Random question selected
- Displayed to user

Why random questions? - Simulates real interview unpredictability - Encourages broader preparation

7.6 Question Bank

- Fetches all questions from database
 - Displays them in a structured list
-

7.7 Admin: Add Question

- Admin-only route
 - Adds new questions to database
 - Accessible only if role == admin
-

8. Database Design

Tables Used

users

Column	Type	Description
id	INTEGER	Primary key
username	TEXT	Unique username
password	TEXT	User password
role	TEXT	admin / aspirant

questions

Column	Type	Description
id	INTEGER	Primary key
question	TEXT	Interview question

9. Security Decisions (Why Certain Things Were Done)

- Admin role not selectable during signup
- Role stored server-side in session
- Database not pushed to GitHub
- Virtual environment ignored using `.gitignore`

10. Deployment-Specific Decisions

requirements.txt

Used so Render knows what to install.

Port Binding

```
app.run(host="0.0.0.0", port=5000)
```

Why? - Cloud platforms require public binding - `127.0.0.1` works only locally

11. Challenges Faced & Resolutions

- Flask CLI issues → solved using `python app.py`
- Git merge conflicts → resolved manually
- Detached HEAD → fixed using branch recovery
- Render port issue → fixed using correct host binding

Each challenge improved debugging and deployment skills.

12. Learning Outcomes

Through this project, the developer learned: - Flask fundamentals - Session management - Role-based access control - Database integration - Git & GitHub workflows - Cloud deployment concepts - Debugging real-world issues

13. Future Enhancements

- Password hashing
 - Timed mock interviews
 - Progress tracking
 - Admin analytics
 - Pagination in question bank
-

14. Final Note

PrepEdge is not just a coding project — it is a **learning artifact** that demonstrates backend understanding, decision-making, and real-world deployment skills.

End of Booklet